



DOCUMENTACIÓN TÉCNICA DEL MVP – TELEMED IA

Plataforma de Telemedicina con Agente Inteligente

Producto Mínimo Viable (MVP)
Derivado del proyecto final TeleMed IA

Sistemas Distribuidos – grupo 2

Código: 82739

Gonzalez Bonilla Jesus Ariel

Ingeniería De Sistemas

Integrantes

María Sofia Aljure Herrera

María Juliana Ferro Bonilla

José Miguel Vera Garzón

Corporación Universitaria Del Huila Corhuila

Septiembre 2026

Neiva - Huila



Contenido

1. Introducción	5
2. ¿Qué es un MVP?	5
3. Relación entre el proyecto final y el MVP.....	5
4. Antecedentes	5
5. Problema que aborda el MVP	6
6. Objetivos	6
6.1. Objetivo general.....	6
6.2. Objetivos específicos.....	6
7. Alcance del MVP	7
7.1. Funcionalidades incluidas en el MVP	7
7.2. Funcionalidades pendientes o fuera del alcance del MVP	7
8. Actores	7
9. Requisitos funcionales y selección del MVP	8
9.1. Correspondencia entre historias de usuario y dominios	9
10. Historias de usuario seleccionadas para el MVP.....	9
11. Flujo principal del MVP	10
12. Tecnologías y herramientas.....	11
13. Arquitectura del MVP	12
14. Dominios y Bounded Contexts del MVP.....	13
14.1 Dominios transversales.....	14
15. Evolución del MVP hacia microservicios.....	14
16. Backend.....	15
16.1 Principales responsabilidades	15
16.2 Seguridad.....	15
16.3 Organización del backend.....	16
16.4 Ejecución	16
17. Frontend	17
17.1 Organización del frontend	17
17.2 Comunicación con el backend	18
17.3 Control de acceso por roles.....	18
18. Estructura general de los proyectos	18
18.1 Estructura del backend	18



18.2	Estructura del frontend	19
19.	Base de datos	20
19.1	Migraciones	20
19.2	Tablas implementadas	21
19.3	Datos iniciales	21
19.4	Verificación de la base de datos.....	22
20.	Flujo de autenticación y roles.....	22
20.1	Administrador	23
20.2	Profesional	23
20.3	Paciente.....	23
20.4	Control de acceso	23
21.	API REST y Swagger	23
21.1	Swagger UI	24
21.2	Principales grupos de endpoints	24
21.3	Evidencia de API.....	25
22.	Pruebas y evidencias	25
22.1	Prueba de ejecución del backend	25
22.2	Prueba de base de datos.....	26
22.3	Prueba de roles.....	26
22.4	Prueba del usuario administrador.....	27
22.5	Prueba de registro de paciente	27
22.6	Prueba de autenticación.....	29
22.7	Prueba del rol ADMIN	30
22.8	Prueba de registro de profesional.....	30
22.9	Resultado general de las pruebas	31
23.	Evidencia de funcionamiento Swagger.....	32
24.	Limitaciones del MVP	32
25.	Trabajo futuro.....	33
26.	Conclusiones	33
27.	Fuentes de referencia	34
Figura 1 Flujo funcional principal del MVP.....		11



Figura 2 Inicio correcto de Spring Boot	26
Figura 3 Prueba de base de datos	26
Figura 4 Prueba de base de datos	26
Figura 5 Prueba de roles en la bd	27
Figura 6 Prueba del usuario administrador	27
Figura 7 Solicitud de registro de paciente	28
Figura 8 Registro 200 OK.....	28
Figura 9 Usuario almacenado en PostgreSQL	28
Figura 10 Registro correspondiente en patients.....	29
Figura 11 Prueba de autenticación	29
Figura 12 Panel de administrador.....	30
Figura 13 Formulario de registro de professional	31
Figura 14 Profesional registrado	31
Figura 15 Funcionamiento Swagger.....	32
Tabla 1 Relación entre el proyecto final y el MVP	5
Tabla 2 Roles Implementados.....	7
Tabla 3 Correspondencia entre historias de usuario y dominios	9
Tabla 4 Historias de usuario seleccionadas para el MVP.....	10
Tabla 5 Tecnologías y herramientas.....	12
Tabla 6 Tablas implementadas en el MVP	21
Tabla 7 Resultado general de las pruebas.....	32

1. Introducción

TeleMed IA es una plataforma académica de telemedicina orientada a facilitar la gestión de la atención médica y la comunicación entre pacientes y profesionales de salud mediante herramientas digitales y un agente inteligente.

El presente documento describe específicamente el Producto Mínimo Viable (MVP) desarrollado para la entrega del primer corte. Este MVP no representa la totalidad del proyecto final: corresponde a una parte del producto, seleccionada para demostrar funcionalidades prioritarias dentro del alcance y tiempo disponibles.

El MVP se implementó como un monolito modular, con backend y frontend, mientras que el proyecto final contempla una evolución más amplia de la solución y su arquitectura.

2. ¿Qué es un MVP?

Un Minimum Viable Product (MVP), o Producto Mínimo Viable, es una versión inicial de un producto que contiene las capacidades esenciales necesarias para entregar valor y demostrar una propuesta con un alcance controlado.

En TeleMed IA, el MVP permite materializar una parte del proyecto final en una solución funcional. No busca implementar los 34 requisitos funcionales ni todas las funcionalidades planteadas inicialmente, sino seleccionar y desarrollar un conjunto alcanzable y demostrable.

3. Relación entre el proyecto final y el MVP

Aspecto	Proyecto final	MVP
Propósito	Construir la visión completa de TeleMed IA.	Demostrar una parte funcional y prioritaria.
Alcance	34 RF, 18 HU y funcionalidades complementarias.	Subconjunto seleccionado del alcance.
Arquitectura	Evolución hacia una solución distribuida/microservicios.	Monolito modular.
Componentes	Evolución completa del sistema.	Backend + frontend + documentación.
Objetivo	Continuar desarrollando el producto completo.	Entregar una versión mínima funcional.

Tabla 1 Relación entre el proyecto final y el MVP

4. Antecedentes

TeleMed IA es un proyecto de telemedicina planteado como una solución progresiva. El proyecto final contempla una plataforma más amplia, mientras que el presente documento se concentra en la versión mínima funcional desarrollada para el MVP.

La definición del producto final establece diferentes áreas de negocio relacionadas con autenticación, pacientes, profesionales, agente inteligente, citas, atención médica, notificaciones y generación de documentos.

Para esta primera versión se seleccionó un subconjunto de funcionalidades que permite demostrar un flujo funcional del sistema sin implementar todavía toda la solución distribuida.

El MVP se desarrolla como un monolito modular, manteniendo una separación interna de responsabilidades y dominios. Esta decisión permite desarrollar y probar las funcionalidades de manera integrada, mientras se conserva una estructura que facilite una futura evolución hacia microservicios independientes.

5. Problema que aborda el MVP

El proyecto identifica dificultades relacionadas con la gestión de la atención médica: algunos procesos pueden requerir gestiones presenciales o generar demoras para obtener una cita, y el profesional puede recibir al paciente sin contar previamente con información organizada sobre el motivo de consulta.

El MVP aborda una parte de este problema mediante una plataforma digital que centraliza funcionalidades de usuarios, perfiles, citas, agenda profesional, atención y resúmenes. La implementación real del agente de IA debe diferenciarse de la experiencia de preconsulta que actualmente aparece como pendiente o placeholder.

6. Objetivos

6.1. Objetivo general

Desarrollar una versión mínima y funcional de TeleMed IA que permita demostrar un flujo principal de gestión de atención médica mediante un backend y un frontend integrados, tomando como base los requisitos priorizados del proyecto final.

6.2. Objetivos específicos

- Implementar registro y autenticación con control de roles.
- Permitir la gestión básica de perfiles.
- Permitir la creación y gestión de citas.
- Permitir al profesional consultar su agenda y gestionar estados de citas.
- Permitir registrar información de la atención médica.
- Generar y consultar los resúmenes disponibles.
- Incorporar la experiencia de preconsulta prevista, diferenciando lo implementado de la IA real pendiente.
- Mantener una base que permita continuar posteriormente hacia el proyecto final.

7. Alcance del MVP

7.1. Funcionalidades incluidas en el MVP

- Backend con Java 17 y Spring Boot 3.3.13.
- Frontend con Angular 17+ e Ionic 7+.
- Autenticación JWT, refresh token y roles.
- Registro de pacientes.
- Gestión de profesionales mediante ADMIN.
- Perfil de paciente.
- Creación, consulta, cancelación y reprogramación de citas.
- Agenda y atención para profesionales.
- Resúmenes clínicos.
- Notificaciones simuladas.
- Swagger/OpenAPI.
- PostgreSQL y Flyway.
- Docker Compose.

7.2. Funcionalidades pendientes o fuera del alcance del MVP

- Implementación real del agente conversacional de IA.
- Envío real de correos mediante SMTP.
- Módulo completo de disponibilidad mediante bloques horarios.
- Descarga de resúmenes en PDF.
- Pruebas unitarias y de integración automatizadas.
- División del MVP en microservicios independientes.

8. Actores

Los roles implementados en el MVP son PACIENTE, PROFESIONAL y ADMIN. El rol ADMIN tiene funciones administrativas de gestión de profesionales y usuarios, mientras que PACIENTE y PROFESIONAL corresponden a los actores principales del proceso de atención.

Actor	Participación en el sistema
Paciente	Registro, autenticación, perfil, gestión de citas y consulta de información disponible.
Profesional de salud	Agenda, revisión de citas y registro de atención.
Administrador	Gestión de usuarios y profesionales.
Agente inteligente	Funcionalidad prevista para preconsulta y organización de información; IA real pendiente según el estado reportado.

Tabla 2 Roles Implementados

9. Requisitos funcionales y selección del MVP

El PDR define 34 requisitos funcionales (RF) relacionados con 18 historias de usuario (HU). Para el MVP se seleccionó un subconjunto de historias de usuario que concentra las funcionalidades necesarias para demostrar los principales flujos de autenticación, gestión de usuarios y profesionales, gestión de perfiles, disponibilidad y gestión de citas.

La siguiente tabla presenta los requisitos funcionales relacionados con las historias de usuario seleccionadas para el MVP y el estado de implementación de cada funcionalidad en esta versión.

RF	Funcionalidad	Estado MVP	Descripción del estado
RF-01– RF-06	Registro, inicio de sesión, roles y cierre de sesión	Incluido	Funcionalidades implementadas para registro, autenticación mediante JWT y control de acceso por roles.
RF-07– RF-08	Recuperación de contraseña	Parcial / simulada	La funcionalidad y su interfaz se encuentran disponibles, pero el proceso de recuperación mediante correo electrónico no está implementado de extremo a extremo.
RF-09– RF-10	Gestión de perfil	Incluido	Gestión del perfil del paciente dentro de las funcionalidades implementadas.
RF-11– RF-12	Gestión de usuarios y profesionales	Incluido	El administrador puede gestionar profesionales mediante las funcionalidades disponibles para su rol.
RF-13– RF-15	Preconsulta con agente inteligente	Parcial / simulada	La experiencia de preconsulta se encuentra disponible, pero la integración con un proveedor de IA real permanece pendiente.
RF-16– RF-20	Disponibilidad y gestión de citas	Parcial	Las funcionalidades de creación, consulta, cancelación y reprogramación de citas están implementadas; la gestión completa de disponibilidad mediante bloques horarios permanece pendiente.
RF-21– RF-24	Agenda profesional	Incluido	El profesional puede consultar y gestionar la información relacionada con su agenda y citas.
RF-25– RF-28	Atención médica	Incluido	Se contemplan las funcionalidades relacionadas con la atención y actualización de estados de las citas.
RF-29– RF-	Resúmenes e historial	Incluido	Se dispone de información relacionada con resúmenes e historial dentro del MVP.

30			
RF-31	Generación/descarga de PDF	Pendiente	La generación y descarga del documento PDF queda como funcionalidad futura.
RF-32– RF-34	Notificaciones	Parcial / simulada	Las notificaciones están contempladas y pueden ser simuladas, pero el envío real mediante correo electrónico no está implementado.

9.1. Correspondencia entre historias de usuario y dominios

Para establecer la trazabilidad del alcance del MVP, las funcionalidades seleccionadas se relacionan con las historias de usuario priorizadas y con los dominios o Bounded Contexts definidos para TeleMed IA. Esta relación permite identificar qué necesidades del usuario son atendidas por el MVP y qué responsabilidad de negocio corresponde a cada funcionalidad.

La correspondencia se organiza de la siguiente manera:

Historia de usuario	Funcionalidad	Dominio / Bounded Context
HU-001	Registro de paciente	Identity & Access
HU-002	Inicio y cierre de sesión	Identity & Access
HU-003	Recuperación de contraseña	Identity & Access
HU-004	Gestión de perfil	Patient Management
HU-005	Gestión de usuarios y profesionales	Professional Management
HU-007	Consultar disponibilidad	Appointment Scheduling
HU-008	Agendar cita	Appointment Scheduling
HU-009	Gestionar mis citas	Appointment Scheduling
HU-010	Consultar agenda médica	Appointment Scheduling / Professional Management
HU-011	Actualizar estado de cita	Appointment Scheduling

Tabla 3 Correspondencia entre historias de usuario y dominios

10. Historias de usuario seleccionadas para el MVP

El MVP se construye a partir de un subconjunto de las historias de usuario definidas para el proyecto final. La selección busca concentrar el desarrollo en las funcionalidades necesarias para demostrar los principales flujos de usuarios, autenticación, gestión de perfiles y gestión de citas.

Las historias seleccionadas para esta versión son:

ID	Historia de usuario
HU-001	Registro de paciente
HU-002	Inicio y cierre de sesión
HU-003	Recuperación de contraseña

HU-004	Gestión de perfil
HU-005	Gestión de usuarios y profesionales
HU-007	Consultar disponibilidad
HU-008	Agendar cita
HU-009	Gestionar mis citas
HU-010	Consultar agenda médica
HU-011	Actualizar estado de cita

Tabla 4 Historias de usuario seleccionadas para el MVP

Estas historias representan el alcance funcional priorizado para el MVP. Las demás historias de usuario permanecen asociadas al proyecto final y podrán incorporarse progresivamente en futuras iteraciones.

11. Flujo principal del MVP

El flujo principal del MVP representa las operaciones esenciales que permiten al paciente registrarse, autenticarse, consultar la disponibilidad de profesionales y gestionar una cita. También contempla las operaciones realizadas posteriormente por el profesional sobre la agenda y el estado de la cita.

El flujo general es el siguiente:

1. **Registro del paciente:** el paciente crea una cuenta proporcionando la información requerida por el sistema.
2. **Inicio de sesión:** el usuario ingresa sus credenciales y el sistema valida la información de acceso y el rol correspondiente.
3. **Gestión del perfil:** el paciente puede consultar y actualizar la información de su perfil.
4. **Consulta de disponibilidad:** el paciente consulta los profesionales y la disponibilidad existente para solicitar una cita.
5. **Selección de profesional y horario:** el paciente selecciona el profesional y el horario disponible que desea reservar.
6. **Creación de la cita:** el sistema valida nuevamente la disponibilidad y registra la cita asociada al paciente y al profesional.
7. **Consulta de agenda médica:** el profesional puede consultar las citas asignadas a su agenda.
8. **Actualización del estado de la cita:** el profesional puede actualizar el estado de la cita de acuerdo con el ciclo definido por el sistema.
9. **Registro de información de atención:** el sistema permite almacenar la información relacionada con la atención y los resúmenes correspondientes.
10. **Consulta de información:** el paciente puede consultar la información relacionada con sus citas y los datos disponibles en el sistema.

Flujo funcional principal del MVP

El siguiente diagrama representa el flujo funcional principal de la versión MVP, desde la autenticación del paciente hasta la gestión de la cita, la atención profesional y la consulta de la información resultante.

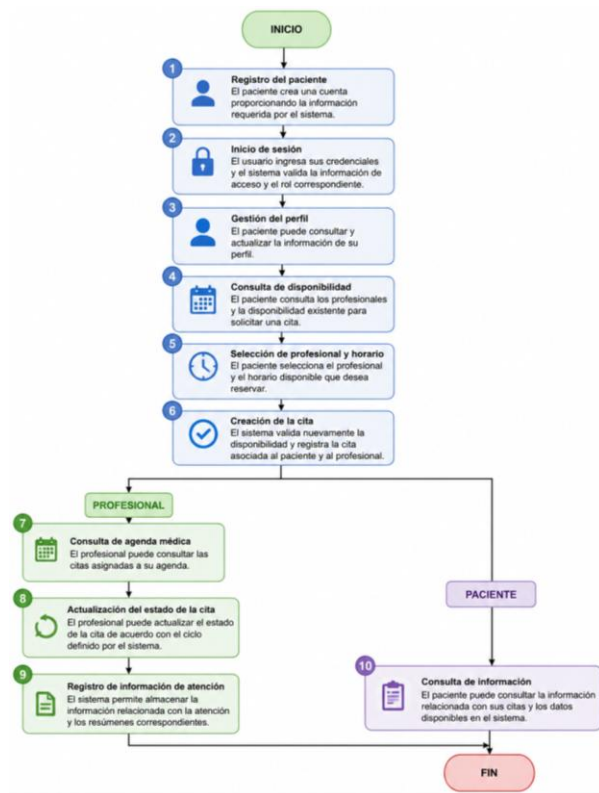


Figura 1 Flujo funcional principal del MVP

12. Tecnologías y herramientas

Capa	Tecnología	Uso
Backend	Java 17	Lenguaje del backend.
Backend	Spring Boot 3.3.13	Framework principal.
Seguridad	Spring Security + JWT + BCrypt	Autenticación y autorización.
Persistencia	PostgreSQL 16 + JPA/Hibernate	Base de datos y ORM.
Migraciones	Flyway	Versionamiento de esquema.
API	OpenAPI 3 + SpringDoc + Swagger UI	Documentación de API.
Frontend	Angular 17+	Aplicación web.
UI/Móvil	Ionic 7+ + Capacitor	Interfaz y soporte móvil.
Frontend	RxJS + Angular Services + Reactive Forms	Estado, servicios y formularios.
Estilos	SCSS	Diseño responsivo.
Monitoreo	Actuator + Micrometer + Prometheus	Monitoreo.
DevOps	Docker + Docker Compose	Contenerización.
Construcción	Maven 3.9+	Construcción backend.
Control de versiones	Git / GitHub	Colaboración y versiones.

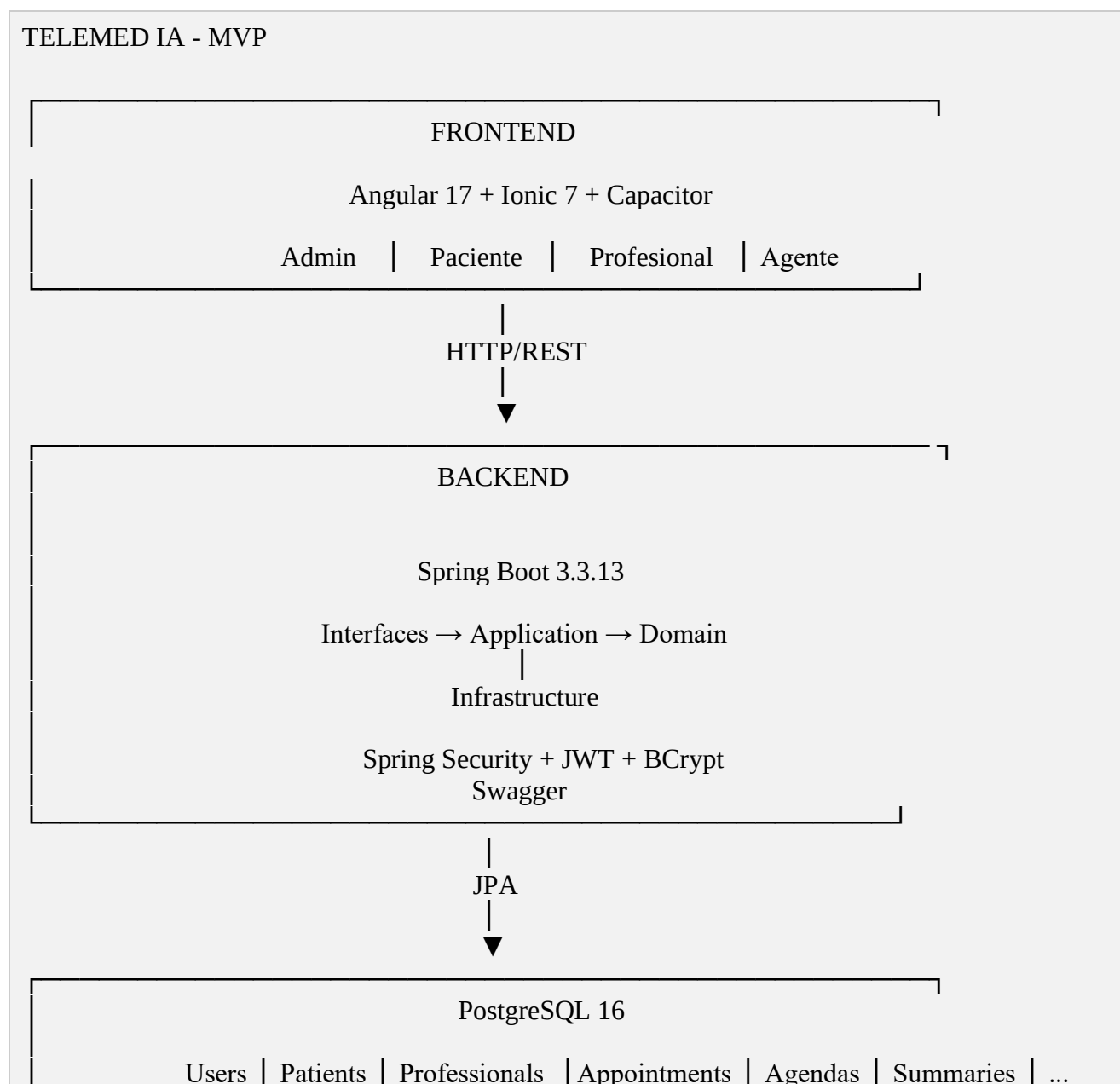
Tabla 5 Tecnologías y herramientas

13. Arquitectura del MVP

El MVP utiliza un monolito modular. El backend está organizado en capas/componentes como application, domain, infrastructure, interfaces y shared; el frontend se organiza mediante core, features y shared.

La modularidad del MVP no implica que cada módulo corresponda actualmente a un microservicio independiente. Los módulos representan límites de responsabilidad que posteriormente podrán evolucionar hacia servicios separados cuando el proyecto avance hacia una arquitectura distribuida.

El siguiente diagrama resume la arquitectura general del MVP:



Flyway

14. Dominios y Bounded Contexts del MVP

Aunque el MVP se implementa como un monolito modular, su estructura se organiza tomando como referencia los dominios y Bounded Contexts definidos para TeleMed IA.

Un dominio representa una parte específica del problema de negocio y concentra las responsabilidades relacionadas con ella. Esta separación permite mantener límites claros dentro del monolito y facilita una posible evolución posterior hacia una arquitectura basada en microservicios.

Para el MVP se consideran los siguientes ocho dominios:

#	Dominio / Bounded Context	Responsabilidad principal	Evolución prevista
1	Identity & Access	Registro, inicio de sesión, recuperación de contraseña, autenticación, JWT y control de acceso.	auth-service
2	Patient Management	Gestión del perfil e información del paciente.	patient-service
3	Professional Management	Gestión del perfil profesional, especialidad y datos del profesional.	professional-service
4	Intelligent Agent	Preconsulta, conversación, preguntas de seguimiento y organización de información.	agent-service
5	Appointment Scheduling	Disponibilidad, agendamiento, cancelación, reprogramación y estados de las citas.	appointment-service
6	Medical Consultation	Registro de la atención, observaciones, diagnóstico, recomendaciones, medicamentos y remisiones.	consultation-service
7	Notifications	Confirmaciones, cancelaciones, reprogramaciones y recordatorios.	notification-service
8	Document Generation	Generación de documentos y resúmenes en PDF.	document-service

La relación entre los dominios y los futuros microservicios se plantea de la siguiente manera:

Dominio → responsabilidad de negocio → posible microservicio futuro

Por ejemplo:

Appointment Scheduling → gestión de citas → appointment-service

En el MVP estos dominios no corresponden a ocho microservicios independientes. Funcionan como módulos internos del monolito, manteniendo sus responsabilidades separadas.

14.1 Dominios transversales

Dentro de los dominios definidos existen capacidades que pueden ser utilizadas por diferentes partes del sistema.

Identity & Access tiene un carácter transversal debido a que diferentes funcionalidades necesitan conocer la identidad, autenticación y permisos del usuario.

Por ejemplo, las funcionalidades de pacientes, profesionales, citas y atención requieren mecanismos de autenticación y autorización.

Notifications también puede apoyar diferentes procesos del sistema, como la creación, cancelación o reprogramación de citas y los recordatorios.

Por esta razón, ambos dominios se consideran candidatos naturales para funcionar posteriormente como capacidades compartidas mediante servicios independientes.

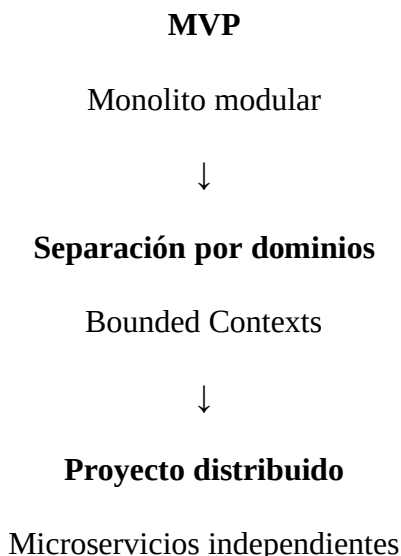
En el MVP estas capacidades permanecen integradas dentro del monolito modular.

15. Evolución del MVP hacia microservicios

El MVP utiliza un monolito modular como estrategia para reducir la complejidad inicial de implementación y facilitar la integración de las funcionalidades seleccionadas.

Sin embargo, los límites de los módulos se definen tomando como referencia los Bounded Contexts establecidos para TeleMed IA. Esta separación permite que, en una etapa posterior, los módulos puedan evolucionar hacia servicios independientes cuando existan necesidades de escalabilidad, despliegue independiente o separación de responsabilidades.

La evolución prevista es:



Por ejemplo:

Appointment Scheduling

→ módulo del monolito en el MVP

→ appointment-service en una futura arquitectura distribuida.

Esta evolución permite que el MVP sea una primera implementación funcional sin adelantar innecesariamente la complejidad propia de una arquitectura de microservicios.

16. Backend

El backend del MVP fue desarrollado utilizando Java 17 y Spring Boot 3.3.13. Su responsabilidad principal es exponer la API REST, implementar la lógica de negocio, gestionar la autenticación y autorización, comunicarse con PostgreSQL y controlar las diferentes funcionalidades de TeleMed IA.

El backend utiliza una arquitectura de monolito modular, organizada por responsabilidades.

16.1 Principales responsabilidades

El backend implementa:

- Registro de pacientes.
- Autenticación de usuarios.
- Generación y validación de tokens JWT.
- Refresh tokens.
- Recuperación de contraseña.
- Control de acceso mediante roles.
- Gestión de pacientes.
- Gestión de profesionales.
- Gestión de citas.
- Gestión de agenda.
- Registro de atención médica.
- Gestión de resúmenes clínicos.
- Gestión de notificaciones.
- Persistencia de información en PostgreSQL.
- Migraciones de base de datos mediante Flyway.
- Documentación de API mediante OpenAPI y Swagger.
- Configuración de CORS.
- Manejo de excepciones.

16.2 Seguridad

La seguridad se implementa mediante Spring Security y JWT.

El sistema utiliza tres roles principales:

- PACIENTE
- PROFESIONAL
- ADMIN

Las solicitudes dirigidas a los endpoints protegidos requieren autenticación mediante un token JWT válido.

Los endpoints relacionados con autenticación, Swagger y health check se encuentran disponibles públicamente.

El backend utiliza BCrypt para el almacenamiento seguro de contraseñas.

16.3 Organización del backend

El backend se encuentra organizado en los siguientes módulos principales:

- application
- domain
- infrastructure
- interfaces
- shared
- config

La capa application contiene servicios y casos de uso de la aplicación.

La capa domain contiene las entidades y elementos relacionados con el dominio del sistema.

La capa infrastructure contiene componentes relacionados con persistencia, correo y otros servicios externos.

La capa interfaces contiene los controladores y puntos de entrada de la API.

La carpeta config contiene configuraciones transversales, como seguridad, CORS y manejo global de excepciones.

16.4 Ejecución

Durante las pruebas locales, el backend fue ejecutado mediante Maven utilizando:

```
mvn spring-boot:run
```

El servidor se inició correctamente en el puerto:

```
8080
```

La ejecución correcta fue comprobada mediante el inicio de Spring Boot y el acceso posterior a Swagger UI.

17. Frontend

El frontend del MVP fue desarrollado utilizando Angular 17 e Ionic 7. La aplicación implementa una interfaz web para los diferentes actores del sistema y consume los servicios REST expuestos por el backend.

Entre las funcionalidades disponibles se encuentran:

- Inicio de sesión.
- Registro de pacientes.
- Manejo de autenticación mediante JWT.
- Control de acceso según el rol del usuario.
- Dashboard principal.
- Gestión del perfil del paciente.
- Consulta y gestión de citas.
- Consulta del historial.
- Preconsulta.
- Gestión de profesionales.
- Gestión de agenda.
- Atención médica.
- Consulta de resúmenes.
- Notificaciones.

La interfaz adapta las funcionalidades disponibles dependiendo del rol autenticado.

17.1 Organización del frontend

El proyecto frontend se encuentra organizado principalmente en las siguientes áreas:

- core: componentes y servicios centrales de la aplicación.
- features: funcionalidades agrupadas por módulo.
- shared: componentes y elementos reutilizables.
- environments: configuración de los diferentes entornos.

Dentro de features se encuentran módulos correspondientes a:

- admin
- agent
- appointment
- auth
- dashboard
- notification
- patient
- professional
- summary

Esta organización permite separar las funcionalidades del sistema y facilita el mantenimiento y evolución del proyecto.

17.2 Comunicación con el backend

El frontend se comunica con el backend mediante una API REST.

Durante el desarrollo local, el backend se encuentra disponible en:

```
http://localhost:8080
```

La API utiliza la ruta base:

```
http://localhost:8080/api
```

El frontend utiliza autenticación mediante tokens JWT para acceder a los recursos protegidos.

17.3 Control de acceso por roles

El frontend adapta las opciones disponibles de acuerdo con el rol autenticado.

Los roles definidos en el sistema son:

- PACIENTE
- PROFESIONAL
- ADMIN

Por ejemplo, el administrador puede acceder al módulo de gestión de profesionales, mientras que el paciente dispone de funcionalidades relacionadas con su perfil, citas e historial.

La autorización también es validada en el backend, evitando que la seguridad dependa únicamente de la interfaz.

18. Estructura general de los proyectos

El MVP está dividido en dos aplicaciones principales:

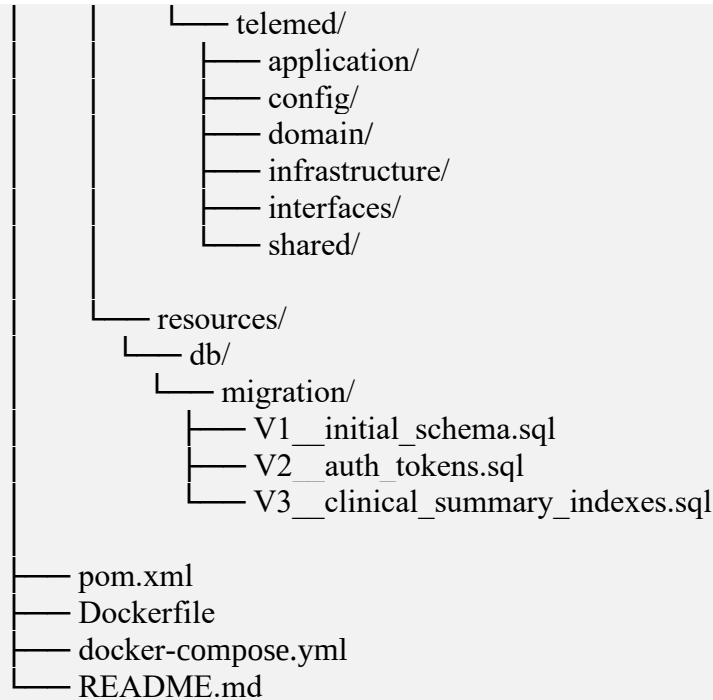
- telemed-backend
- telemed-frontend

Esta separación permite mantener diferenciadas la lógica del servidor y la interfaz de usuario.

18.1 Estructura del backend

La estructura general del backend es:

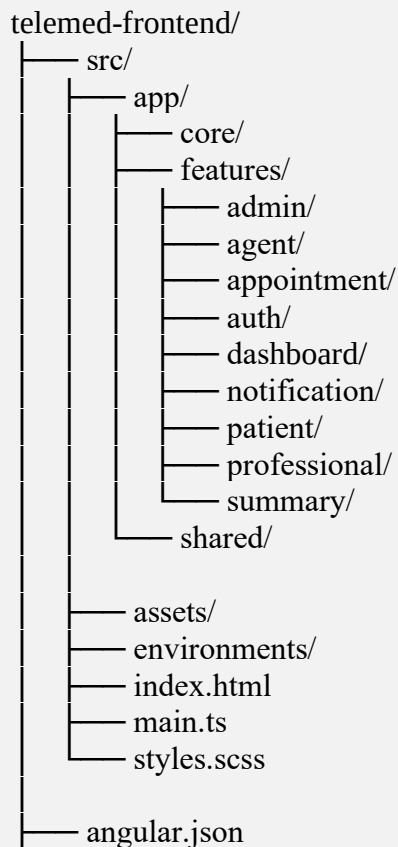
```
telemed-backend/  
├── src/  
│   ├── main/  
│   └── java/  
│       └── com/  
└──
```



La estructura refleja la organización modular del backend y permite separar responsabilidades.

18.2 Estructura del frontend

La estructura general del frontend es:



- package.json
- ionic.config.json
- capacitor.config.ts

Esta organización permite separar las funcionalidades por módulos y reutilizar componentes y servicios comunes.

19. Base de datos

El MVP utiliza PostgreSQL como sistema gestor de base de datos.

La base de datos utilizada durante las pruebas locales se denomina:

telemed

El acceso se realiza mediante el usuario:

telemed

El backend utiliza Flyway para controlar y versionar las migraciones de la estructura de la base de datos.

19.1 Migraciones

Las migraciones se encuentran en:

src/main/resources/db/migration/

Actualmente se encuentran las siguientes versiones:

- V1__initial_schema.sql
- V2__auth_tokens.sql
- V3__clinical_summary_indexes.sql

La migración V1 contiene el esquema inicial de la aplicación, incluyendo las entidades principales del sistema.

La migración V2 contiene las estructuras relacionadas con tokens de autenticación y recuperación de contraseña.

La migración V3 incorpora índices adicionales para mejorar el rendimiento de determinadas consultas y una restricción de unicidad para las agendas profesionales.

Flyway registra las migraciones ejecutadas en la tabla:

flyway_schema_history

19.2 Tablas implementadas

El esquema de PostgreSQL contiene las tablas correspondientes a los principales módulos del MVP, incluyendo usuarios, roles, pacientes, profesionales, agendas, disponibilidad, citas, conversaciones, mensajes, resúmenes, notificaciones y tokens de autenticación.

Adicionalmente, Flyway utiliza la tabla flyway_schema_history para mantener el control de las migraciones ejecutadas. Esta tabla pertenece al mecanismo de versionamiento de la base de datos y no representa una funcionalidad de negocio del sistema.

Tabla	Función
roles	Roles del sistema
specialties	Especialidades médicas
users	Usuarios
patients	Información de pacientes
professionals	Información de profesionales
agendas	Agenda de profesionales
availability_blocks	Bloques de disponibilidad
appointments	Citas médicas
conversations	Conversaciones
messages	Mensajes
preconsultation_summaries	Resúmenes de preconsulta
attention_summaries	Resúmenes de atención
post_summaries	Resúmenes posteriores
notifications	Notificaciones
refresh_tokens	Tokens de renovación
password_reset_tokens	Tokens de recuperación
flyway_schema_history	Control de migraciones

Tabla 6 Tablas implementadas en el MVP

El MVP contempla la funcionalidad de recuperación de contraseña mediante la interfaz correspondiente y estructuras de soporte para tokens de recuperación. Sin embargo, el envío real del correo electrónico y la validación completa del flujo de recuperación se consideran funcionalidades parciales/simuladas para esta versión.

19.3 Datos iniciales

La base de datos contiene información inicial para permitir las pruebas del sistema.

Entre los datos iniciales se encuentran los roles:

- PACIENTE
- PROFESIONAL
- ADMIN

También se incluyen especialidades médicas como:

- Medicina General

- Pediatría
- Dermatología
- Cardiología

Además, el sistema crea un usuario administrador inicial para facilitar la configuración y las pruebas del MVP.

19.4 Verificación de la base de datos

La existencia de las tablas fue comprobada directamente desde PostgreSQL mediante el comando:

```
\dt
```

El resultado mostró las 17 relaciones correspondientes al esquema del MVP.

Como evidencia se debe incluir una captura de la consola de PostgreSQL mostrando el resultado de \dt.

También se recomienda incluir capturas de consultas sobre:

```
SELECT id, name FROM roles;
```

y:

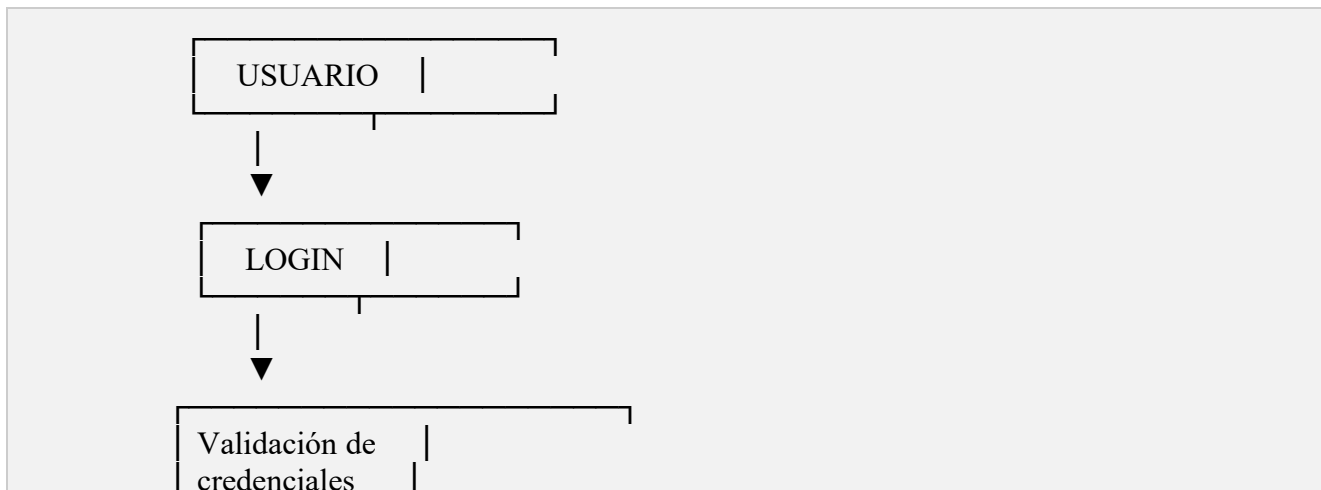
```
SELECT id, full_name, email, identity_document, role_id, verified, active  
FROM users;
```

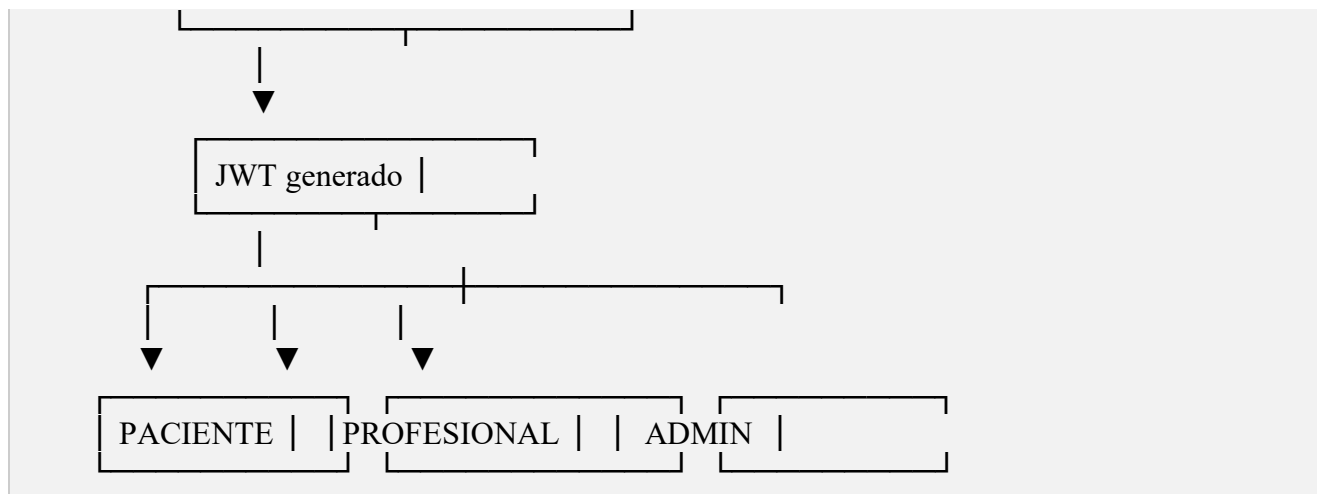
Estas consultas permiten comprobar los roles y usuarios existentes durante las pruebas.

20. Flujo de autenticación y roles

TeleMed IA implementa un esquema de autenticación basado en JWT y control de acceso mediante roles.

El flujo general es:





El backend valida el token recibido en las solicitudes protegidas y determina el rol del usuario.

20.1 Administrador

El administrador tiene permisos para gestionar usuarios y profesionales.

Durante las pruebas se verificó el acceso del administrador al módulo de profesionales del frontend.

Desde este módulo el administrador puede registrar profesionales.

20.2 Profesional

El profesional corresponde al usuario encargado de las actividades relacionadas con la atención médica.

Una vez registrado por el administrador, el profesional dispone de sus propias credenciales y puede autenticarse de forma independiente.

20.3 Paciente

El paciente puede registrarse desde el sistema y posteriormente iniciar sesión.

El paciente dispone de funcionalidades relacionadas con:

- Perfil.
- Citas.
- Preconsulta.
- Historial.
- Resúmenes.
- Notificaciones.

20.4 Control de acceso

Los endpoints protegidos requieren autenticación.

21. API REST y Swagger

El backend expone una API REST para la comunicación con el frontend.

La API se encuentra disponible localmente mediante:

`http://localhost:8080/api`

La documentación interactiva fue implementada utilizando OpenAPI 3 y SpringDoc.

21.1 Swagger UI

Durante las pruebas locales, Swagger UI estuvo disponible en:

`http://localhost:8080/swagger-ui/index.html`

Swagger permite visualizar los endpoints disponibles, consultar sus métodos HTTP, revisar los parámetros y cuerpos de solicitud y ejecutar las operaciones directamente contra el backend.

21.2 Principales grupos de endpoints

Los principales endpoints REST del MVP se organizan de acuerdo con los dominios funcionales de la aplicación:

Autenticación

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/refresh
- POST /api/auth/password-reset

Pacientes

- GET /api/patients/{id}
- PUT /api/patients/{id}

Profesionales

- GET /api/professionals
- POST /api/professionals
- POST /api/professionals/admin/register
- GET /api/professionals/{id}

Citas

- POST /api/appointments
- PATCH /api/appointments/{id}/status

Resúmenes

- POST /api/summaries/preconsultation

Notificaciones

- POST /api/notifications
- GET /api/notifications/user

Los endpoints anteriores representan las principales operaciones REST utilizadas por las funcionalidades seleccionadas para el MVP. La documentación detallada de cada operación se encuentra disponible mediante Swagger UI.

21.3 Evidencia de API

Se utilizará Swagger como una de las principales evidencias técnicas del funcionamiento del backend.

Se deben incluir capturas de:

1. Swagger UI mostrando los controladores disponibles.
2. Registro de un paciente mediante POST /api/auth/register.
3. Respuesta exitosa del registro.
4. Login mediante POST /api/auth/login.
5. Ejecución de operaciones de la API mediante Swagger UI.

Postman puede utilizarse adicionalmente para repetir las pruebas de API, pero no es indispensable cuando Swagger permite ejecutar y demostrar correctamente los endpoints.

22. Pruebas y evidencias

Las pruebas del MVP se realizaron verificando la comunicación entre frontend, backend y base de datos.

Se utilizaron:

- Aplicación web del frontend.
- Spring Boot.
- PostgreSQL.
- Swagger UI.
- Consola de PostgreSQL.

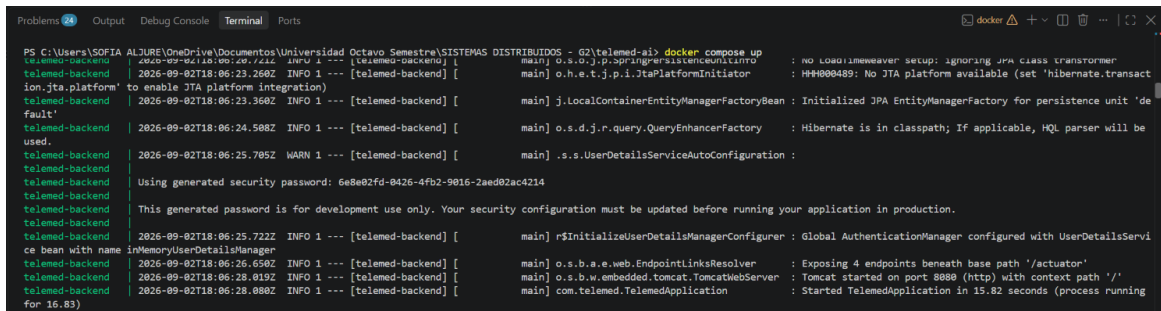
22.1 Prueba de ejecución del backend

Se ejecutó el backend mediante Maven:

```
mvn spring-boot:run
```

El servidor inició correctamente en el puerto 8080.

La ejecución correcta fue verificada mediante los mensajes de inicio de Spring Boot y posteriormente mediante el acceso a Swagger UI.



```
PS C:\Users\SOFIA ALJURE\OneDrive\Documentos\Universidad Octavo Semestre\SISTEMAS DISTRIBUIDOS - G2\telemed-ai> docker compose up
telemed-backend | 2026-09-02T18:06:23.260Z INFO 1 --- [telemed-backend] [main] o.s.o.j.p.spr.spring.persistence.integration-jta.platform' to enable JTA platform integration)
telemed-backend | 2026-09-02T18:06:23.360Z INFO 1 --- [telemed-backend] [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
telemed-backend | 2026-09-02T18:06:24.508Z INFO 1 --- [telemed-backend] [main] o.s.d.j.r.query.QueryEnhancerFactory : Hibernate is in classpath; If applicable, HQL parser will be used.
telemed-backend | 2026-09-02T18:06:25.785Z WARN 1 --- [telemed-backend] [main] .s.s.UserDetailsServiceAutoConfiguration :
telemed-backend | Using generated security password: 6e8e02fd-0426-4fb2-9016-2aed02ac4214
telemed-backend | This generated password is for development use only. Your security configuration must be updated before running your application in production.
telemed-backend | 2026-09-02T18:06:25.722Z INFO 1 --- [telemed-backend] [main] r$initializeUserDetailsServiceConfigurer : Global AuthenticationManager configured with UserDetailsServiceImpl bean with name inMemoryUserDetailsService
telemed-backend | 2026-09-02T18:06:26.650Z INFO 1 --- [telemed-backend] [main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 4 endpoints beneath base path '/actuator'
telemed-backend | 2026-09-02T18:06:28.019Z INFO 1 --- [telemed-backend] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
telemed-backend | 2026-09-02T18:06:28.080Z INFO 1 --- [telemed-backend] [main] com.telemed.TelemedApplication : Started TelemedApplication in 15.82 seconds (process running for 16.83)
```

Figura 2 Inicio correcto de Spring Boot

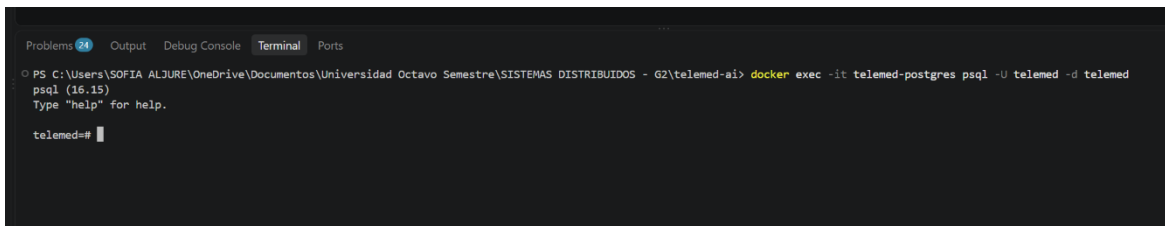
22.2 Prueba de base de datos

Se verificó la conexión a PostgreSQL mediante el usuario telemed.

Posteriormente se ejecutó:

```
\dt
```

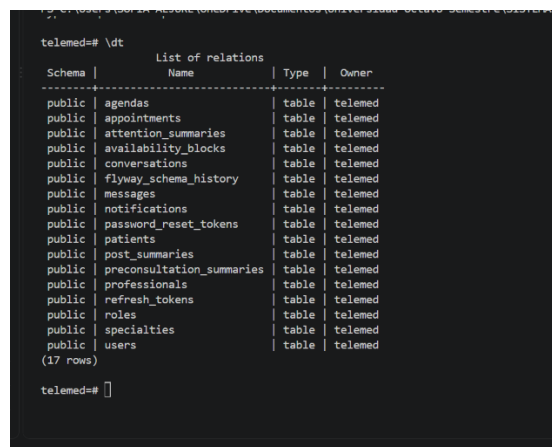
El resultado mostró 17 tablas correspondientes al esquema del MVP.



```
PS C:\Users\SOFIA ALJURE\OneDrive\Documentos\Universidad Octavo Semestre\SISTEMAS DISTRIBUIDOS - G2\telemed-ai> docker exec -it telemed-postgres psql -U telemed -d telemed psql (16.15)
Type \help for help.

telemed=#
```

Figura 3 Prueba de base de datos



```
telemed=# \dt
          List of relations
Schema | Name                | Type  | Owner
-----|-----|-----|-----
public | agendas              | table | telemed
public | appointments         | table | telemed
public | attention_summaries  | table | telemed
public | availability_blocks  | table | telemed
public | conversations        | table | telemed
public | flyway_schema_history | table | telemed
public | messages             | table | telemed
public | notifications        | table | telemed
public | password_reset_tokens | table | telemed
public | patients             | table | telemed
public | post_summaries       | table | telemed
public | preconsultation_summaries | table | telemed
public | professionals        | table | telemed
public | refresh_tokens       | table | telemed
public | roles                | table | telemed
public | specialties          | table | telemed
public | users                | table | telemed
(17 rows)

telemed=#
```

Figura 4 Prueba de base de datos

22.3 Prueba de roles

Se verificaron los roles mediante:

```
SELECT id, name FROM roles;
```

Resultado esperado:

1 | PACIENTE
2 | PROFESIONAL
3 | ADMIN

```
telemed=# SELECT id, name FROM roles;
 id |  name
-----+-----
  1 | PACIENTE
  2 | PROFESIONAL
  3 | ADMIN
(3 rows)

telemed=#
```

Figura 5 Prueba de roles en la bd

22.4 Prueba del usuario administrador

Se verificó la existencia del usuario administrador mediante:

```
SELECT id, full_name, email, identity_document, role_id, verified, active
FROM users
WHERE email = 'admin@email.com';
```

El usuario aparece asociado al rol ADMIN.

```
telemed=# SELECT id, full_name, email, identity_document, role_id, verified, active
telemed=# FROM users
telemed=# WHERE email = 'admin@email.com';
 id | full_name | email | identity_document | role_id | verified | active
-----+-----+-----+-----+-----+-----+-----
  1 | Admin | admin@email.com | 000000000 | 3 | t | t
(1 row)

telemed=#
```

Figura 6 Prueba del usuario administrador

22.5 Prueba de registro de paciente

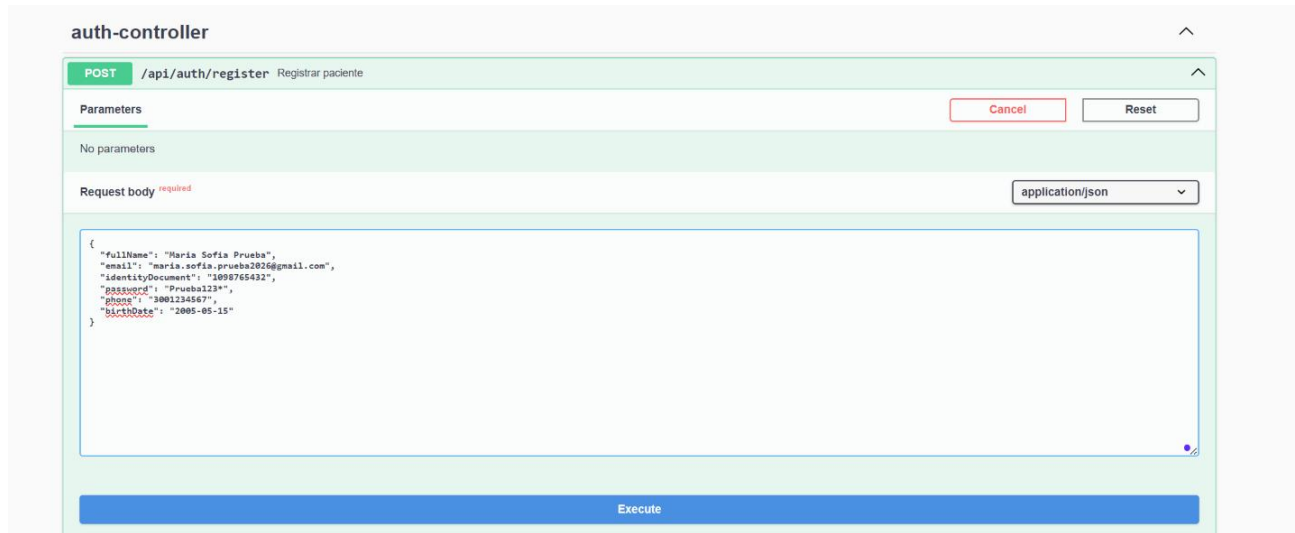
Se ejecutó el endpoint:

POST /api/auth/register

utilizando Swagger.

La operación respondió con HTTP 200 OK y generó las credenciales de autenticación correspondientes.

Posteriormente se verificó en PostgreSQL que el usuario y su perfil de paciente fueron almacenados correctamente.



auth-controller

POST /api/auth/register Registrar paciente

Parameters

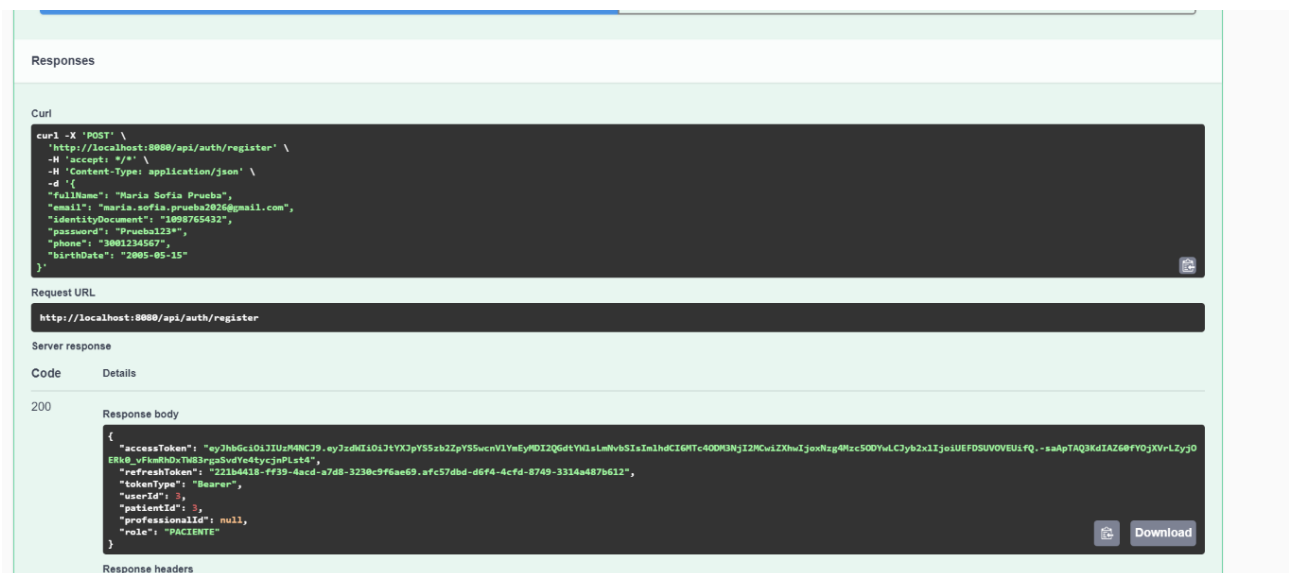
No parameters

Request body *required* application/json

```
{
  "fullName": "Maria Sofia Prueba",
  "email": "maria.sofia.prueba2026@gmail.com",
  "identityDocument": "1098765432",
  "password": "Prueba123*",
  "phone": "3001234567",
  "birthDate": "2005-05-15"
}
```

Execute

Figura 7 Solicitud de registro de paciente



Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/auth/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "fullName": "Maria Sofia Prueba",
    "email": "maria.sofia.prueba2026@gmail.com",
    "identityDocument": "1098765432",
    "password": "Prueba123*",
    "phone": "3001234567",
    "birthDate": "2005-05-15"
  }'
```

Request URL

http://localhost:8080/api/auth/register

Server response

Code	Details
200	Response body <pre>{ "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bm90cmVudCI6IjIwMjYyMDUyMjYyIiwiaWF0IjoiMTY1MjYyMDUyMjYyIn0", "refreshToken": "221b4418-ff39-4acd-a7d8-3230c9f6a69.afc57dbd-d6f4-4cfd-8749-3314a487b612", "tokenType": "Bearer", "userId": 3, "patientId": 3, "professionalId": null, "role": "PACIENTE" }</pre> Response headers

Figura 8 Registro 200 OK

```
telemed=# SELECT * FROM users WHERE email = 'maria.sofia.prueba2026@gmail.com';
 id | full_name | email | identity_document | role_id | password_hash | verified | active |
-----+-----+-----+-----+-----+-----+-----+-----+
  3 | Maria Sofia Prueba | maria.sofia.prueba2026@gmail.com | 1098765432 | 1 | $2a$10$v8x5LZwaxKL0f6jY5QOpAurmxzqNjI1Lb91SPS3t2XWnejl22v36 | f | t | 2026-09-02 19:11:00.529881+00 |
(1 row)

telemed=#
```

Figura 9 Usuario almacenado en PostgreSQL

```
telemed=# SELECT * FROM patients WHERE id = 3;
 id | birth_date |   phone   | medical_history | description
-----+-----+-----+-----+-----
  3 | 2005-05-15 | 3001234567 |                 |
(1 row)
```

```
telemed=#
```

Figura 10 Registro correspondiente en patients

22.6 Prueba de autenticación

Se probó el endpoint:

POST /api/auth/login

La autenticación permite obtener un accessToken y un refreshToken.

El token de acceso se utiliza posteriormente para realizar solicitudes protegidas.

Request body

required

application/json

```
{  
  "email": "maria.sofia.prueba2026@gmail.com",  
  "password": "Prueba123*"  
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  
  'http://localhost:8080/api/auth/login' \  
  -H 'accept: */*' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "email": "maria.sofia.prueba2026@gmail.com",  
    "password": "Prueba123*"  
  }'
```

Request URL

http://localhost:8080/api/auth/login

Server response

Code

Details

200

Response body

```
{  
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpYSSJ9.eyJySSZyb2ZpYS5wcmlmeSjDI2Q6dtYwlsLmVibSIsIm1hdCI6MTc4ODM3NzY0SwlZXhwIjoxNDg4PzgxdjMSLCJyb2xiIjoieUEFDSUV0VEUiFQ.Qo56oBt5-PDKBYLq3NVJANLS",  
  "refreshToken": "24734595-58ff-43ac-9163-e3d75cf8550e.f69ad680-8946-4bb4-a129-f18d26e7953f",  
  "tokenType": "Bearer",  
  "userId": 1,  
  "patientId": 1,  
  "professionalId": null,  
}
```

Figura 11 Prueba de autenticación

22.7 Prueba del rol ADMIN

Se inició sesión con el usuario administrador.

El frontend mostró correctamente el usuario:

admin@email.com
ADMIN

y permitió acceder al módulo de administración de profesionales.

Desde este módulo se dispone de la opción:

REGISTRAR PROFESIONAL

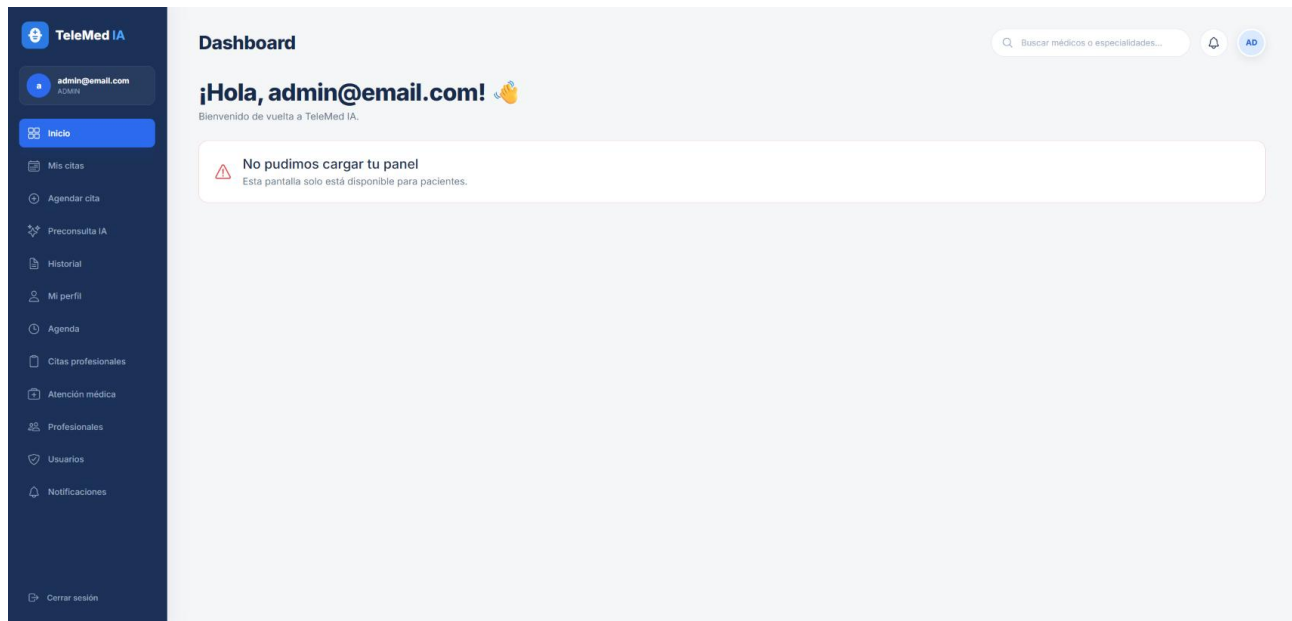


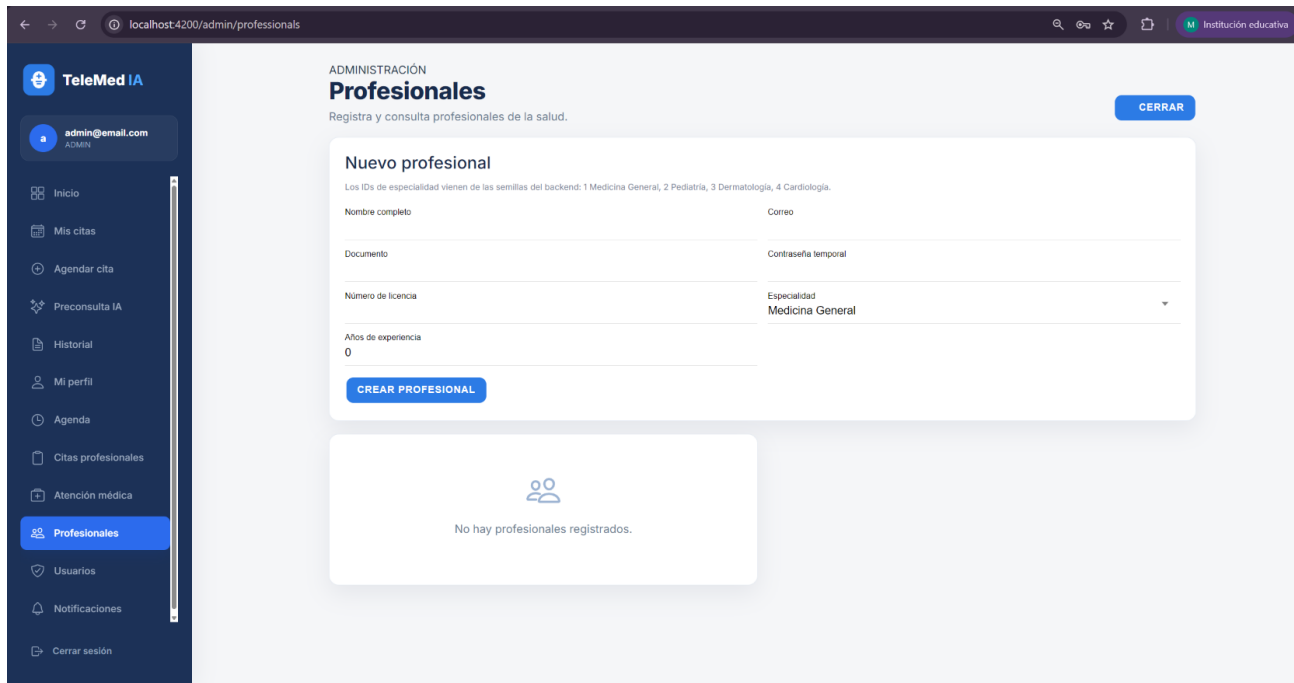
Figura 12 Panel de administrador

22.8 Prueba de registro de profesional

El administrador puede registrar un profesional mediante el módulo correspondiente.

El profesional registrado queda asociado a un usuario y a una especialidad médica.

Una vez creado, el profesional puede autenticarse con sus propias credenciales y utilizar las funcionalidades correspondientes a su rol.



ADMINISTRACIÓN
Profesionales
Registra y consulta profesionales de la salud.

Nuevo profesional
Los IDs de especialidad vienen de las semillas del backend: 1 Medicina General, 2 Pediatría, 3 Dermatología, 4 Cardiología.

Nombre completo:

Correo:

Documento:

Contraseña temporal:

Número de licencia:

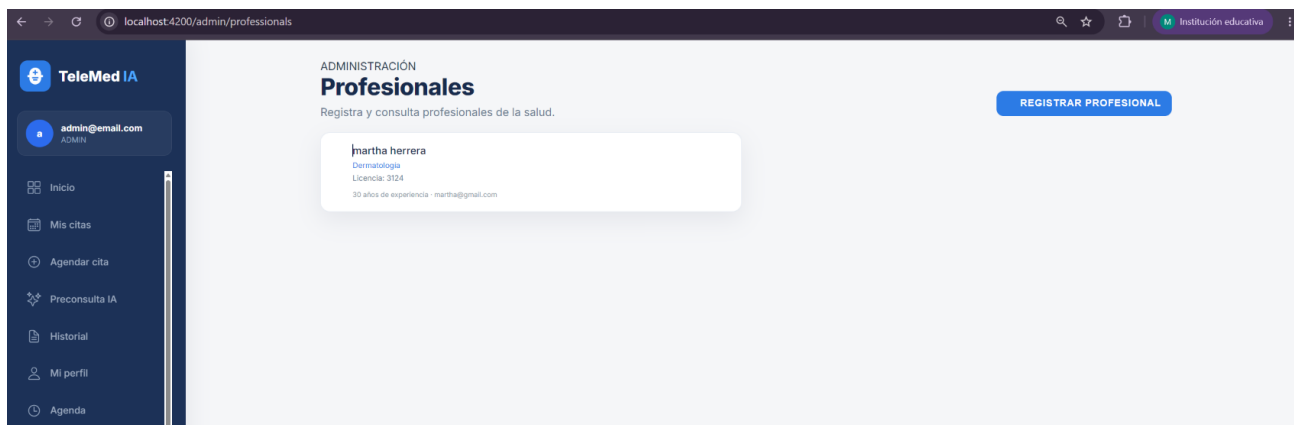
Especialidad:

Años de experiencia:

CREAR PROFESIONAL

No hay profesionales registrados.

Figura 13 Formulario de registro de profesional



ADMINISTRACIÓN
Profesionales
Registra y consulta profesionales de la salud.

REGISTRAR PROFESIONAL

martha herra
Dermatología
Licencia: 3124
30 años de experiencia - martha@gmail.com

Figura 14 Profesional registrado

22.9 Resultado general de las pruebas

Las pruebas realizadas permitieron comprobar:

Prueba	Resultado
Inicio del backend	Correcto
Conexión con PostgreSQL	Correcta
Migraciones Flyway	Correctas
Creación de tablas	Correcta
Roles	Correctos
Usuario administrador	Correcto
Registro de paciente	Correcto
Registro de paciente en BD	Correcto
Login	Correcto

Generación de JWT	Correcta
Protección de endpoints	No evidenciada en esta versión
Swagger UI	Correcto
Acceso del administrador	Correcto
Gestión de profesionales	Disponible
Frontend	Funcional

Tabla 7 Resultado general de las pruebas

Las pruebas anteriores corresponden a verificaciones realizadas durante la ejecución local del MVP.

23. Evidencia de funcionamiento Swagger

Evidencia Swagger UI

Descripción: documentación interactiva de la API REST mediante Swagger UI.

Dirección: <http://localhost:8080/swagger-ui/index.html#/>

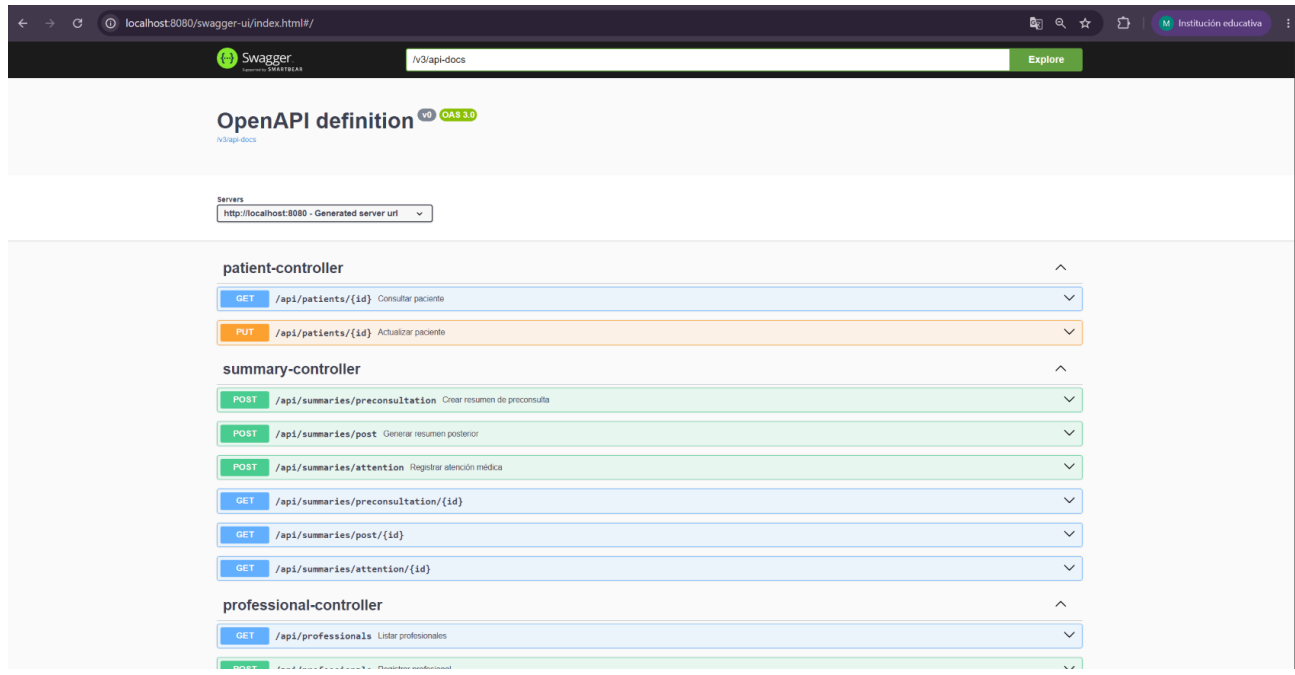


Figura 15 Funcionamiento Swagger

24. Limitaciones del MVP

Aunque el MVP presenta un flujo funcional de los principales componentes de TeleMed IA, todavía existen funcionalidades previstas para el proyecto completo que no forman parte de esta versión.

Las principales limitaciones son:

- El agente conversacional de inteligencia artificial todavía no representa una implementación completa de IA clínica.

- La preconsulta inteligente se encuentra limitada a la funcionalidad actualmente implementada en el frontend y backend.
- El envío de correos electrónicos reales mediante SMTP no se encuentra completamente implementado.
- La disponibilidad mediante bloques horarios requiere continuar su desarrollo.
- La generación y descarga de documentos PDF se mantiene como funcionalidad futura.
- Las pruebas automatizadas unitarias y de integración requieren ampliación.
- El MVP continúa utilizando una arquitectura de monolito modular.
- La separación en microservicios corresponde a una evolución prevista para el proyecto final.

25. Trabajo futuro

A partir de la versión actual del MVP se plantea continuar con la evolución de TeleMed IA mediante:

- Implementación completa del agente conversacional de inteligencia artificial.
- Integración del agente con el proceso de preconsulta.
- Implementación de preguntas de seguimiento y generación automática del resumen previo.
- Integración del agente con la consulta de disponibilidad y gestión de citas.
- Implementación de comunicación por correo electrónico.
- Ampliación del sistema de disponibilidad de profesionales.
- Generación y descarga de resúmenes en PDF.
- Ampliación de pruebas automatizadas.
- Fortalecimiento de las pruebas de seguridad.
- Mejoramiento de las notificaciones y recordatorios.
- Incorporación progresiva de las funcionalidades restantes del proyecto final.
- Evolución progresiva del monolito modular hacia una arquitectura distribuida.
- Separación de los Bounded Contexts en microservicios independientes cuando las necesidades del sistema lo justifiquen.

26. Conclusiones

El MVP de TeleMed IA permitió transformar una parte de los requisitos definidos para el proyecto final en una aplicación funcional compuesta por frontend, backend y base de datos.

Durante la implementación se logró integrar Angular e Ionic en el frontend con un backend desarrollado en Java y Spring Boot, utilizando PostgreSQL como sistema gestor de base de datos.

La utilización de Flyway permitió controlar la creación y evolución del esquema de la base de datos mediante migraciones versionadas.

La implementación de Spring Security y JWT permitió establecer mecanismos de autenticación y autorización, diferenciando los roles de PACIENTE, PROFESIONAL y ADMIN.



Las pruebas realizadas mediante Swagger, PostgreSQL y la interfaz web permitieron comprobar el funcionamiento de diferentes componentes del MVP, incluyendo el registro de usuarios, autenticación, generación de JWT, persistencia de información y gestión administrativa de profesionales. La protección de endpoints se encuentra implementada mediante Spring Security y JWT, aunque su prueba específica no se incluye como evidencia en esta versión.

El MVP también establece una base técnica para continuar desarrollando las funcionalidades restantes del proyecto final, especialmente la incorporación completa del agente inteligente y la evolución hacia una arquitectura distribuida.

27. Fuentes de referencia

- Product Requirements Document (PDR) – AI Telemedicine Chatbot Platform (TeleMed IA), agosto de 2026.
- Propuesta de proyecto – Plataforma de Telemedicina con Agente Inteligente, agosto de 2026.
- Documento interno “¿Qué es TeleMed IA?”.
- README técnico actual de TeleMed IA.
- Repositorio de documentación telemed-ia-docs.